

Structure of Java Programming

class: 3

1. Packages:

→ Packages are nothing but folders (to divide code accordingly)

→ Syntax:

```
package package name;
```

Ex: package com.film.db; (folder in folder).
(package name)

→ It creates ~~com~~ db folder inside the film folder, and film folder in com folder.

→ Package name should be outside the class.

Ex: package com.film;

```
public class Test {
```

```
}
```

2. Class:

→ It is a type of blue print (plan before coding).

→ Blueprint Implementation is an object [()] → Instance of class.

→ Every .java file is a class.

→ Syntax: public class Test {
→ [we have to write anything in the class block only].
}

3. Main Method:

→ Without main method, we can compile the code but it will not run the code.

→ Firstly, JVM will search for main method.

→ Main method should be inside the class.

Syntax: public static void main (String[] args) {
(args)
}
public static void main (String args[]) {
}

Variables: Used to store some data (like containers to store our grocery).

Ex: age = 27 ;

name = Gayatri;
↓
Variables.

→ Static, Non-Static & Local Variables.

Methods: Used to do some tasks (Functions).

→ to reuse code (used).

Syntax: public void method () {
(empty) (method name)
}

Import: To import objects from one file to another file.

• packages com.flm [we have to give package name also],

import com.flm.Test2

import com.* (all classes imports).

* We cannot import any file from default package.

* To rename that packages, we have to use package com(packagename);

Object:

* Non-static variables must have to create objects.

* Local & static variables need, to create objects.
not

* Syntax:

(classname)

Test t = new Test ();
(object) (class name).

t.name ();

Structure:

1. Package name

2. Import packages & class.

3. Class [public class Test { }

4. Main method [public static void main (String [] args { }

→ Static variables — objects not required.

→ Non-Static / Instance variables — objects required.

→ Local variables — objects not required.

* // → In Java, we used for commenting.

→ Methods [public void method () { }

Naming Conventions

1. Class Names:

- pascal case → Start with capital letters.
- Ex: Chennai Super Kings x
Chennai Super Kings ✓
- Every word in a class starts with capital letters.
- Class names donot start with numbers.
- Java allows only two special characters (-, \$).

Allowed by Java: Alphabets, -, \$.

Not-allowed by Java: Numbers, any other special characters.

→ 'Space' also not allowed by Java.

Ex: Fayaz Flm → Java allows this but not a proper approach to java - Not recommended. ~~followed words:~~

2. Variables:

- Camel Case type → 1st letter is small letter and remaining following words of 1st letter is in capital].

Ex: chennai Super Kings.

→ Allowed : Alphabets, -, \$ (in the starting of class name).

→ Not allowed : Numbers, any other special characters.

→ We can use that both special characters (-, \$) at a time.

Ex: fayaz 1 ✓

1 fayaz x.

3. Method names:

→ Camel Case

→ Allowed : Alphabets, -, \$.

→ Not allowed : Numbers, any other special characters.

Exs 1. fayaz Flm () { } 2) Fayaz Flm () { }.

* packages:

→ small case / lower case (space not allowed in java).

Ex: com.film.db.

* Project Names:

→ Pascal Case.

* Starting with number & any special character other than (- and \$), will not allow in Java.

DATA TYPES

→ We should tell Java, what type of variable we are creating: (i.e. Numbers, characters, string, true/false, decimal).

→ Datatypes are of 2 types:

1. Primitive Datatype.

2. Non-primitive Datatype.

1. Primitive Datatypes: 8 types.

* 1 GB = 1024 MB
* 1 MB = 1024 KB
* 1 KB = 1024 bytes
* 1 byte = 8 bits.

Number storage types:

a) byte - 1 byte - 8 bits - allows numbers b/w -2^7 to 2^7-1 i.e., -128 to 128.

b) Short - 2 bytes - 16 bits - allows numbers b/w -2^{15} to $2^{15}-1$ i.e., -32768 to 32767.

c) int - 4 bytes - 32 bits - allows numbers b/w -2^{31} to $2^{31}-1$.

d) long - 8 bytes - 64 bits - allows numbers b/w -2^{63} to $2^{63}-1$.

Decimal numbers:

e) float - 4 bytes - precision - accept 7 digits after point i.e. decimal.
Ex: 0.12345678 f.

f) double - 8 bytes - precision - accept 16 digits after point i.e. decimal.
Ex: 0.12345678912345678.

→ Normal numbers i.e., non-decimal numbers will also accept.

g) Char - Characters - 2 bytes

ex: 'A', 'B', '\$', '+', ...

* We can also pass alphabets, numbers, special characters in 'char'.

h) boolean - true / false - 1 bit of memory (negligible memory).

* These 8 datatypes are not classes.

2. Non-primitive data types: 2 types - These are classes.

→ User defined data types / classes.

a) String

b) Array [].

Ex:

```
package com.datatypes;
```

```
public class Test {
```

```
    public static void main (String [ ] args) {
```

```
        byte byteValue = 127;
```

```
        short shortValue = 32767;
```

```
        int intValue = 999999999;
```

```
        long longValue = 9999999999999L;
```

```
        float floatValue = 1.0035f / f(8) 1.123456789f;
```

```
        double doubleValue = 1.123456789123456789;
```

```
        char character = 'a';
```

```
        boolean booleanValue = true / false.
```

```
    }
```

```
        System.out.println (* All)
```

```
    }
```

```
}
```

Output: 127

32767

999999999

9999999999999

1.1234568

1.1234567891234568

a

true / false.

* Default Values:

* For all numbers, Java will give default value '0'.

* Default value of float = 0.0.

1) byte = 0

5) float = 0.0

2) Short = 0

6) double = 0.0

3) int = 0

7) char = - (empty character)

4) long = 0

8) boolean = false.

14/04/2025

a) String:

→ Group of characters. accepts, stores in double quotes (" ").

Ex: String Company = "FLM" → 6 bytes (3 x 2 bytes = 6 bytes)

→ Single characters also can be passed in string.

→ Each character carry 2 bytes of memory.

→ Overhead - 20-30 bytes storage [extra added to the string] (along with it)

OPERATORS

{ $a + b$ }
↑ operators
↓ operands

1. Arithmetic Operators [+, -, *, /, % (modulus operator)]

2. Relational Operators.

3. Assignment Operators [=] [+=, -=, *=, /=, %=]

4. Unary Operators.

5. Logical Operators.

1. Arithmetic Operators:

+ = Addition

- = Subtraction

* = Multiplication

/ = Division - gives quotient always - Output: 5

% = Modulo - return only remainder. - Output: 1

Ex:
$$\begin{array}{r} 2 \overline{) 11} \\ \underline{10} \\ 1 \end{array}$$

Addition:

Ex: package project 2;

```
public class Arithmetic Operators {
```

```
    public static void main (String [] args) {
```

```
        int a = 5;
```

```
        int b = 20;
```

```
        int c = 4;
```

```
        int d = 2;
```

```
        int addition = a+b;
```

```
        System.out.println ("a+b = " + addition);
```

```
    }
```

```
}
```

output: ~~25~~

a+b = 25

[implicit type casting

float addition = a+b]

Concatenation
↓
Concatenation {
String + —
}

* Concatenation: addition of any two variables / string / characters etc.

Ex: String Company = "FLM"

String institution = "Edutech"

```
System.out.println (Company + " " + institution);
```

output: FLM Edutech.

Concatenation.

* When we add string and number, & by default it convert into String.

Ex: System.out.println ("ab" + a + "cd");

output: ab5cd.

→ string + number = string.

Ex: System.out.println ("ab" + a + b); → Output: ab520

System.out.println (a+b + "ab"); → Output: 25ab.

⇒ [Always operation done from left to right]

Subtraction:

```
System.out.println (b-a);
```

output: 15
-15

```
System.out.println (a-b);
```

* We cannot subtract a variable from string in Java.

* Integer - float = float.

* Multiplication: *

System.out.println(a * b); → Output: 100

* We can't multiply a string with variables.

* Division: /

System.out.println(b/a); → Output: 24

System.out.println(a/c); → Output: 1

* Modulo: %

System.out.println(a % c); Output = 1

System.out.println(c % a); Output = 4

Ex: float f = 12.5 f;
float f = 8.5 f.

System.out.println(e + f); → Output: 21.0
4.0

System.out.println(e - f);

⇒ In java, float divides exactly i.e. with decimal values.

* Float always returns float value.

System.out.println(e % f);

⇒ $\begin{matrix} e = 7.0 \\ f = 2.0 \end{matrix}$

System.out.println(e / f); Output: 1.0
3.5

Assignment Operators: (=)

Variable = operation/variable.

1) int a = 5 2) a = a + 2 // 5 + 2
a = 7

a) System.out.println(a); Output: 5

a = a + 2 // a + 2

System.out.println(a); Output: 7

a += 2

System.out.println(a); Output: 9

a -= 4

System.out.println(a); Output: 4

a *= 5

System.out.println(a); Output: 20